



南開大學
Nankai University

计算机学院
计算机体系结构实验报告

TLB & Cache

姓名：林晖鹏

学号：2312966

专业：计算机科学与技术

2026 年 1 月 9 日

目录

1 实验要求	3
2 实验原理	3
2.1 TLB 原理	3
2.2 Cache 原理	4
3 TLB 设计与实现	4
3.1 整体功能	4
3.2 双端口查找设计	5
3.3 命中与翻译	5
3.4 写端口 (TLBWI) 实现	6
3.5 读端口 (TLBR) 实现	6
3.6 实验结果	7
4 CPU 的 TLB 集成	7
4.1 流水总线扩展与 CP0 地址宏定义	7
4.2 TLB 在 CPU 顶层的集成	8
4.3 TLB 指令添加	9
4.3.1 TLB 指令译码	9
4.3.2 TLBP 指令的查找实现	9
4.3.3 TLB 指令的精确提交	10
4.3.4 TLBWI 写表项实现	10
4.4 TLB 相关寄存器的实现	10
4.5 实验结果	12
5 Cache 模块实现	13
5.1 总体设计与配置	13
5.2 Cache 存储阵列设计	13
5.3 Cache 接口与握手语义	13
5.4 Cache 状态机设计	14
5.5 Cache 查找阶段 (S_LOOKUP)	14
5.6 Cache 缺失与替换策略	15
5.6.1 替换策略	15
5.6.2 脏块写回 (S_WB)	16
5.7 Cache 行填充 (Refill)	16
5.7.1 读请求阶段 (S_RD_REQ)	16
5.7.2 等待返回 (S_RD_WAIT)	16
5.7.3 行写入与写分配 (S_REFILL)	17
5.8 写策略与字节写支持	18
5.8.1 Write-back 与 Write-allocate	18
5.8.2 字节写掩码处理	18
5.9 请求锁存与单请求处理模型	18

5.10 实验结果	18
6 icache & dcache 集成 CPU	19
6.1 总体结构概述	19
6.2 段属性判定	19
6.2.1 设计动机	19
6.2.2 实现方式	20
6.3 ICache / DCache Wrapper 设计	20
6.3.1 设计目标	20
6.3.2 uncached 返回不可门控问题	20
6.3.3 ICache Wrapper 关键逻辑	20
6.3.4 DCache Wrapper 特点	21
6.4 Cache Burst 仲裁与顶层集成	21
6.4.1 仲裁状态机	21
6.5 AXI Bridge 对 Cache Burst 的支持	23
6.5.1 设计扩展	23
6.5.2 避免 Cache 返回被反压	23
6.6 关键点总结	23
6.7 实验结果	23
6.8 性能实验分析	24
7 实验总结	24

1 实验要求

1. 请参考《CPU 设计实战》第九章和第十章的内容，在现有的五级流水线 CPU 的基础上，增加 TLB 模块和 cache 模块。
2. TLB 部分推荐完成 TLB 模块设计和 TLB 相关指令和 CP0 寄存器部分，例外支持部分自己把握。
3. Cache 部分至少设计出 ICache 和 DCache，并尝试在 CPU 中集成测试，其他部分大家自己把握。

2 实验原理

2.1 TLB 原理

在采用虚拟存储机制的处理器中，程序使用的地址为**虚拟地址 (Virtual Address, VA)**，而实际访问存储器时需要使用**物理地址 (Physical Address, PA)**。虚拟地址到物理地址的映射关系由操作系统维护在页表中，但页表通常存放在内存中，若每次访存都访问页表将带来较大的性能开销。

为此，处理器中通常引入 **TLB (Translation Lookaside Buffer)**，作为页表项的高速缓存，用于加速虚拟地址到物理地址的转换过程。

TLB 的基本工作流程如下：

1. CPU 产生虚拟地址，将其拆分为 **虚拟页号 (VPN)** 与 **页内偏移 (Offset)**
2. 使用 VPN (以及 ASID) 在 TLB 中进行查找
3. 若命中，则直接得到物理页号 (PFN) 与访问属性
4. 将 PFN 与 Offset 拼接，形成最终的物理地址

若 TLB 未命中或访问属性不满足要求，通常需要触发异常并由操作系统处理。在本实验中，为简化设计，未实现 TLB 异常和例外的处理，而且，当未命中时暂时退化为直接使用虚拟地址作为物理地址。

(1) 全相联 TLB

全相联 (Fully Associative) TLB 结构，即每个表项都可能与任意虚拟页号匹配。这种设计在硬件上需要并行比较所有表项，但由于实验中 TLB 项数较小 (16 项)，实现简单且命中延迟可接受。

全相联 TLB 的命中条件为：

$$VPN2_{tlb} = VPN2_{req} \wedge (G = 1 \vee ASID_{tlb} = ASID_{req})$$

(2) 双页结构 (Dual Page Structure)

按照 MIPS 架构规范，每个 TLB 表项对应 **两个连续虚拟页**，共用同一个 VPN2 字段，通过虚拟地址中的 odd_page 位区分奇偶页。

双页结构的特点如下：

- 一个表项存储两个 PFN (PFN0 / PFN1) 及其属性
- 表项数减少一半，降低比较规模

- 查找命中后，根据页内地址第 12 位选择对应页

在 TLB 命中后，依据 `odd_page` 信号选择对应的 PFN 及访问属性，并完成地址拼接。

(3) 多端口查找支持

为了支持五级流水 CPU 中取指与访存阶段的并行工作，TLB 需要支持多端口并发查找。

- `s0` 端口：用于取指阶段的地址翻译
- `s1` 端口：用于访存阶段，同时复用于 TLBP 指令

两个端口逻辑独立、可在同一周期并行访问 TLB，从而避免结构冲突。

2.2 Cache 原理

在处理器系统中，主存访问延迟较高，若每次取指或访存都直接访问主存，将严重限制处理器性能。因此通常在 CPU 与主存之间引入 **Cache**，以利用程序的 **时间局部性**和 **空间局部性**。

Cache 的基本工作流程如下：

1. CPU 发出物理地址访问请求
2. Cache 判断是否命中
3. 命中则直接返回数据
4. 未命中则访问下一级存储并进行填充

在采用虚拟存储机制的系统中，Cache 通常工作在 **物理地址**上，因此地址转换顺序一般为：

虚拟地址 → TLB → 物理地址 → Cache

CPU 侧通过 `inst_sram` 与 `data_sram` 接口访问 Cache。

此外，由于 Cache 访问具有一定延迟，在流水线发生 flush（如 TLB 更新或异常）时，可能仍有尚未返回的数据请求。因此可能需要引入 **discard 机制**，丢弃 flush 之前发出的 Cache 返回数据，以保证流水线状态的一致性。

3 TLB 设计与实现

按照指导书的顺序来，我们先来完成 lab13 的内容，lab13 给了一个 tlb 的测试框架。按照指导书给出的输入输出架构，下面讲解我是如何实现这个 TLB 的：

3.1 整体功能

按照指导书所说，我们要实现了一个**全相联 TLB**，用于虚拟地址到物理地址的转换，并支持多端口并行访问，满足取指与访存阶段的需求：

- **两个独立查找端口**
 - `s0_*`：主要用于取指阶段的地址翻译
 - `s1_*`：主要用于访存阶段，同时复用于 TLBP 指令

- 一个写端口：we + w_*, 用于 TLBWI 指令
- 一个读端口：r_index + r_*, 用于 TLBR 指令

模块参数 TLBNUM 用于指定 TLB 表项数量（指导书上指定为 16），索引位宽通过 $\$clog2(TLBNUM)$ 自动计算。

3.2 双端口查找设计

这里设计了两个端口供查找，s0_* 与 s1_* 两个查找端口逻辑完全独立，可以在同一周期并行工作。其中端口 1 的 s1_index 常用于 TLBP 指令，用于向上层返回匹配表项的索引。

两个端口的设计相同，查找方式如下小节所述。

3.3 命中与翻译

这里不过多讲解接口设计，直接来讲如何运作的。对于任意表项 i ，命中条件定义为：

$$match_i = (tlb_vpn2[i] = s_vpn2) \wedge (tlb_g[i] \vee (tlb_asid[i] = s_asid))$$

即虚拟页号必须匹配，且满足全局表项或 ASID 相等。

我们这里先初始化命中结果状态，然后对整个表进行遍历，根据上面的命中条件进行筛选；注意这里没有对多命中的情况进行处理，在正常维护下不应该出现重复命中，如果真的不巧出现了，那么会返回最后一次命中的结果。

根据指导书给的接口，这里我们采用双页结构。双页结构能在同样的 TLB 区域大小时，减少表项数，能加快查找的速度。

当表项命中后，根据 odd_page 信号选择输出页，并对命中结果进行对应赋值。

```

1  always @(*) begin
2      // 初始状态：未命中，索引置0，属性置默认值
3      s1_found_r  = 1'b0;
4      s1_index_r  = {($clog2(TLBNUM)) {1'b0}};
5      s1_pfn_r    = 20'd0;
6      s1_c_r      = 3'd0;
7      s1_d_r      = 1'b0;
8      s1_v_r      = 1'b0;
9
10     // 遍历所有TLB表项，进行并行查找匹配
11     for (i = 0; i < TLBNUM; i = i + 1) begin
12         // 匹配条件：表项VPN2一致 + （全局位有效 或 ASID一致）
13         if (tlb_vpn2[i] == s1_vpn2 && (tlb_g[i] || (tlb_asid[i] == s1_asid))) begin
14             s1_found_r  = 1'b1;          // 置位命中标志
15             s1_index_r  = i[$clog2(TLBNUM)-1:0]; // 记录命中索引（供TLBP指令使用）
16
17             // 根据奇数页标志选择对应属性（0选pfn0/c0/d0/v0，1选pfn1/c1/d1/v1）
18             if (s1_odd_page == 1'b0) begin
19                 s1_pfn_r = tlb_pfn0[i];
20                 s1_c_r   = tlb_c0[i];
21                 s1_d_r   = tlb_d0[i];

```

```

22         sl_v_r    = tlb_v0[i];
23     end else begin
24         sl_pfn_r   = tlb_pfn1[i];
25         sl_c_r     = tlb_c1[i];
26         sl_d_r     = tlb_d1[i];
27         sl_v_r     = tlb_v1[i];
28     end
29 end
30 end
31 end

```

3.4 写端口 (TLBWI) 实现

写端口在时钟上升沿工作，用于实现 TLBWI 指令的语义：

- 当 `we = 1` 时，将 `w_*` 信号写入 `w_index` 指定的表项
- 当 `we = 0` 时，TLB 内容保持不变

```

1  always @(posedge clk) begin
2      if (we) begin // 写使能有效时执行写入操作
3          tlb_vpn2[w_index] <= w_vpn2;
4          tlb_asid[w_index] <= w_asid;
5          tlb_g[w_index] <= w_g;
6          tlb_pfn0[w_index] <= w_pfn0;
7          tlb_c0[w_index] <= w_c0;
8          tlb_d0[w_index] <= w_d0;
9          tlb_v0[w_index] <= w_v0;
10         tlb_pfn1[w_index] <= w_pfn1;
11         tlb_c1[w_index] <= w_c1;
12         tlb_d1[w_index] <= w_d1;
13         tlb_v1[w_index] <= w_v1;
14     end
15 end

```

需要注意的是，这里未实现复位或初始化逻辑。在仿真阶段，若某些表项从未被写入，其内容可能为不确定值 (X)，从而影响查找结果。需要在启动阶段对 TLB 进行初始化或逐项填充。

3.5 读端口 (TLBR) 实现

读端口为纯组合逻辑，用于支持 TLBR 指令：

- 输入： `r_index`
- 输出：对应索引表项的全部字段 `r_*`

```

1  assign r_vpn2 = tlb_vpn2[r_index];
2  assign r_asid = tlb_asid[r_index];

```

```

3  assign r_g      = tlb_g[r_index];
4  assign r_pfn0   = tlb_pfn0[r_index];
5  assign r_c0     = tlb_c0[r_index];
6  assign r_d0     = tlb_d0[r_index];
7  assign r_v0     = tlb_v0[r_index];
8  assign r_pfn1   = tlb_pfn1[r_index];
9  assign r_c1     = tlb_c1[r_index];
10 assign r_d1     = tlb_d1[r_index];
11 assign r_v1     = tlb_v1[r_index];

```

该端口不引入额外时序延迟。

3.6 实验结果

lab13 提供了一个测试仿真文件，这里直接放结果图：

```

INFO: [USF-XSim-96] XSim completed. Design snapshot 'testbench_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
> launch_simulation: Time (s): cpu = 00:00:09 ; elapsed = 00:00:22 . Memory (MB): peak = 1194.391 ; gain = 26.785
> run all
OK!!!write
OK!!!search
OK!!!read
=====
Test end!
----PASS!!!
> $finish called at time : 5615 ns : File "C:/Users/lhppp/Downloads/lab/tlb_verify/testbench/testbench.v" Line 63

```

图 3.1: lab13 实验结果

4 CPU 的 TLB 集成

上节已经完成了 TLB 模块本身的设计与实现。本节在此基础上，按照实验指导书要求，将 TLB 集成进五级流水 CPU，并新增 TLB 相关指令 (TLBP、TLBWI、TLBR) 以及对应的 ‘Index’、‘EntryHi’、‘EntryLo0’、‘EntryLo1’ CP0 寄存器，从而实现基本的虚拟地址翻译流程。

需要说明的是，这里没有实现异常例外。当 TLB 未命中或页属性不满足访问要求时，当前实现会退化为 “不进行翻译，直接使用虚拟地址作为物理地址”。

4.1 流水总线扩展与 CP0 地址宏定义

为在流水线中正确传递 TLB 指令控制信息，需要扩展多级流水寄存器之间的总线宽度。同时，为支持 CP0 中新增的 TLB 相关寄存器，在头文件中补充相应的地址宏定义。

Listing 1: 流水总线宽度扩展与 CP0 地址定义

```

1  // ds_to_es: 增加 TLBR/TLBWI/TLBP 3 个指令位
2  define DS_TO_ES_BUS_WD 214// es_to_ms / ms_to_ws: 透传 3 个 TLB 指令位 + tlbp
   found,indexdefine ES_TO_MS_BUS_WD 171
3  define MS_TO_WS_BUS_WD 132// CP0 addr = rd[15:11],
   sel[2:0]define CP0_INDEX_ADDR 8'h00
4  define CP0_ENTRYLO0_ADDR 8'h10define CP0_ENTRYLO1_ADDR 8'h18
5  define CP0_ENTRYHI_ADDR 8'h50

```


4.2 TLB 在 CPU 顶层的集成

在之前的 CPU 流水线测试中，给出的虚拟地址直接就是对应的物理地址，所以不需要翻译。但是在实际情况下，往往虚拟地址并不对应物理地址，所以需要 TLB 作为中间部件进行翻译。

要翻译的就是两种地址，一个是指令地址，需要翻译成 **inst rom** 里面的物理地址，一个是访存地址，需要翻译为 **data ram** 里面的物理地址：

取指 虚拟地址由 **pre_IF_stage** 产生，原本直接输出到 SRAM 的接口，现在需要经过 TLB 进行地址翻译，再输出到外部指令 SRAM 接口 **inst_sram_addr**。

将虚拟地址接入 TLB，得到翻译成功信号后，对于 kseg0/kseg1 地址段采用直接映射，否则在 TLB 命中且页属性满足条件时使用 PFN 拼接物理地址；如果不成功，直接用原本的虚拟地址

```
1 // vaddr->paddr translate
2 wire inst_direct_map;
3 assign inst_direct_map = (inst_sram_vaddr[31:29] == 3'b100) ||
    (inst_sram_vaddr[31:29] == 3'b101);
4 wire inst_tlb_ok;
5 assign inst_tlb_ok = tlb_s0_found & tlb_s0_v;
6 assign inst_sram_addr = inst_direct_map ? {3'b000, inst_sram_vaddr[28:0]} :
7     (inst_tlb_ok ? {tlb_s0_pfn, inst_sram_vaddr[11:0]} :
        inst_sram_vaddr);
```

访存 虚拟地址由 **EXE_stage** 产生，同样在 TLB 完成翻译，再送至 **data_sram_addr**。

```
1 // data vaddr->paddr translate (same rule as inst)
2 wire data_direct_map;
3 assign data_direct_map = (data_sram_vaddr[31:29] == 3'b100) ||
    (data_sram_vaddr[31:29] == 3'b101);
4 wire data_tlb_ok;
5 assign data_tlb_ok = tlb_s1_found & tlb_s1_v & (~data_sram_wr | tlb_s1_d);
6 assign data_sram_addr = data_direct_map ? {3'b000, data_sram_vaddr[28:0]} :
7     (data_tlb_ok ? {tlb_s1_pfn,
        data_sram_vaddr[11:0]} :
        data_sram_vaddr);
```

discard 机制 为避免 flush 发生后错误使用 flush 之前发出的取指返回数据，在取指阶段引入 discard 机制。仅当返回数据未被标记为 discard 时，才允许其进入流水线。

这里在流水线流程各位置中加入了 flush 信号传递，然后判断是否 dataok。

```
1 always @ (posedge clk) begin
2     if (reset) begin
3         inst_sram_discard <= 2'b00;
4     end else if (ws_ex || ws_eret || ws_tlb_flush) begin
5         inst_sram_discard <= {pfs_inst_waiting, fs_inst_waiting};
6     end else if (inst_sram_data_ok) begin
7         if (inst_sram_discard == 2'b11) begin
8             inst_sram_discard <= 2'b01;
```

```

9      end else if (inst_sram_discard == 2'b01) begin
10          inst_sram_discard <= 2'b00;
11      end else if (inst_sram_discard == 2'b10) begin
12          inst_sram_discard <= 2'b00;
13      end
14  end
15 end
16 assign inst_sram_data_ok_discard = inst_sram_data_ok && ~|inst_sram_discard;

```

4.3 TLB 指令添加

4.3.1 TLB 指令译码

这里需要在译码阶段新增对 TLBP、TLBWI 和 TLBR 指令的识别。其中, TLBP 指令依赖 CP0.EntryHi 的内容, 而流水线中可能存在尚未提交的 MTC0 EntryHi 指令。由于未实现前递机制 (指导书中说太麻烦了, 性价比不高), 此处采用阻塞方式处理该数据相关。

Listing 2: TLB 指令译码与 TLBP 阻塞逻辑

```

1 assign inst_tlbwr = op_d[6'h10] & rs_d[5'h10] & func_d[6'h01] & rd_d[5'h00] &
  rt_d[5'h00] & sa_d[5'h00];
2 assign inst_tlbwi = op_d[6'h10] & rs_d[5'h10] & func_d[6'h02] & rd_d[5'h00] &
  rt_d[5'h00] & sa_d[5'h00];
3 assign inst_tlbp = op_d[6'h10] & rs_d[5'h10] & func_d[6'h08] & rd_d[5'h00] &
  rt_d[5'h00] & sa_d[5'h00];
4
5 wire tlbwr_entryhi_block = ds_valid && inst_tlbwr && (es_mtc0_entryhi ||
  ms_mtc0_entryhi);
6 assign ds_ready_go = !(mfc0_block || tlbwr_entryhi_block);

```

4.3.2 TLBP 指令的查找实现

TLBP 指令在 EXE 阶段复用 TLB 的 s1 查找端口, 以 CP0.EntryHi 中的 VPN2 和 ASID 作为查找关键字。查找结果 (是否命中及命中索引) 随流水线继续向后传递。

Listing 3: TLBP 查找 key 生成与结果透传

```

1 assign tlb_s1_vpn2 = es_inst_tlbwr ? cp0_entryhi[31:13] :
  es_data_vaddr[31:13];
2 assign tlb_s1_odd_page = es_inst_tlbwr ? 1'b0 : es_data_vaddr[12];
3 assign tlb_s1_asid = cp0_entryhi[7:0];
4
5 assign es_tlbwr_found = es_valid && es_inst_tlbwr && tlb_s1_found;
6 assign es_tlbwr_index = tlb_s1_index;
7
8 assign es_mtc0_entryhi_o = es_valid && es_inst_mtc0 &&
  (es_cp0_addr == CP0_ENTRYHI_ADDR);
9

```

4.3.3 TLB 指令的精确提交

TLBP、TLBR 和 TLBWI 指令均在 WB 阶段精确提交。其中，TLBR 和 TLBWI 会改变 TLB 或 CP0 状态，因此在提交时需要触发一次流水线刷新，以保证后续取指和访存使用最新的翻译结果。

Listing 4: TLB 指令提交与流水线刷新

```
1 assign ws\_tlbp\_commit = ws\_valid && ws\_inst\_tlbp && !ws\_ex;
2 assign ws\_tlbr\_commit = ws\_valid && ws\_inst\_tlbr && !ws\_ex;
3 assign ws\_tlbwi\_commit = ws\_valid && ws\_inst\_tlbwi && !ws\_ex;
4
5 assign ws\_tlb\_flush = ws\_tlbr\_commit || ws\_tlbwi\_commit;
6 assign ws\_tlb\_flush\_pc = ws\_pc + 32'h4;
```

4.3.4 TLBWI 写表项实现

TLBWI 指令在 WB 阶段提交时，根据 CP0.Index、EntryHi、EntryLo0 和 EntryLo1 的内容，向 TLB 写入指定索引的表项。表项的全局位 G 按 MIPS 规范由 EntryLo0.G 与 EntryLo1.G 共同决定。

Listing 5: TLBWI 写 TLB 表项

```
1 assign tlb_we = ws\_tlbwi\_commit;
2 assign tlb_w_index = ws\_cp0\_index[3:0];
3 assign tlb_w_vpn2 = ws\_cp0\_entryhi[31:13];
4 assign tlb_w_asid = ws\_cp0\_entryhi[7:0];
5 assign tlb_w_g = ws\_cp0\_entrylo0[0] & ws\_cp0\_entrylo1[0];
```

4.4 TLB 相关寄存器的实现

CP0 中新增 Index、EntryHi、EntryLo0 和 EntryLo1 寄存器。需要注意的是，这里，其中 Index 的 P 位仅由 TLBP 指令更新，这里一开始没有保护，然后第一个测试点一直没过，做了一下午 debug 没发现。mtc0 Index 仅写索引字段。

Listing 6: CP0 中 TLB 相关寄存器更新逻辑

```
1 wire mtc0\_index = mtc0\_we && (cp0\_addr == CP0\_INDEX\_ADDR); wire mtc0\_entryhi =
  mtc0\_we && (cp0\_addr == CP0\_ENTRYHI\_ADDR);
2 wire mtc0\_entrylo0 = mtc0\_we && (cp0\_addr == CP0\_ENTRYLO0\_ADDR); wire
  mtc0\_entrylo1 = mtc0\_we && (cp0\_addr == CP0\_ENTRYLO1\_ADDR);
3
4 always @(posedge clk) begin
5   if (rst) begin
6     c0\_index <= 32'b0;
7     c0\_entryhi <= 32'b0;
8     c0\_entrylo0 <= 32'b0;
9     c0\_entrylo1 <= 32'b0;
10  end else begin
11    // mtc0 writes (mask to the fields we actually implement)
12    if (mtc0\_index) begin
13      // c0\_index[31] <= cp0\_wdata[31]; // 删除或注释掉这一行，保护 P 位
```

```

14         c0\_index[3:0] <= cp0\_wdata[3:0];
15         c0\_index[30:4] <= 27'b0;
16     end
17     if (mtc0\_entryhi) begin
18         c0\_entryhi[31:13] <= cp0\_wdata[31:13];
19         c0\_entryhi[12:8] <= 5'b0;
20         c0\_entryhi[7:0] <= cp0\_wdata[7:0];
21     end
22     if (mtc0\_entrylo0) begin
23         c0\_entrylo0[31:26] <= 6'b0;
24         c0\_entrylo0[25:0] <= {cp0\_wdata[25:6], cp0\_wdata[5:3], cp0\_wdata[2],
25             cp0\_wdata[1], cp0\_wdata[0]};
26     end
27     if (mtc0\_entrylo1) begin
28         c0\_entrylo1[31:26] <= 6'b0;
29         c0\_entrylo1[25:0] <= {cp0\_wdata[25:6], cp0\_wdata[5:3], cp0\_wdata[2],
30             cp0\_wdata[1], cp0\_wdata[0]};
31     end
32     // tlbp updates Index (precise commit)
33     if (tlbp\_we) begin
34         c0\_index[31] <= ~tlbp\_found;
35         c0\_index[30:4] <= 27'b0;
36         c0\_index[3:0] <= tlbp\_index;
37     end
38     // tlbr updates EntryHi/Lo0/Lo1 (precise commit)
39     if (tlbr\_we) begin
40         c0\_entryhi[31:13] <= tlbr\_vpn2;
41         c0\_entryhi[12:8] <= 5'b0;
42         c0\_entryhi[7:0] <= tlbr\_asid;
43
44         c0\_entrylo0[31:26] <= 6'b0;
45         c0\_entrylo0[25:0] <= {tlbr\_pfn0, tlbr\_c0, tlbr\_d0, tlbr\_v0,
46             tlbr\_g};
47
48         c0\_entrylo1[31:26] <= 6'b0;
49         c0\_entrylo1[25:0] <= {tlbr\_pfn1, tlbr\_c1, tlbr\_d1, tlbr\_v1,
50             tlbr\_g};
51     end
52 end

```

4.5 实验结果

到这里，TLB 就集成到 CPU 上了，这里放 lab14 的测试结果：

```
=====
Test begin!
----[ 21555 ns] Number 8'd01 Functional Test Point PASS!!!
      [ 22000 ns] Test is running, debug_wb_pc = 0xbfc00770
      [ 32000 ns] Test is running, debug_wb_pc = 0xbfc00d2c
----[ 37215 ns] Number 8'd02 Functional Test Point PASS!!!
      [ 42000 ns] Test is running, debug_wb_pc = 0xbfc005d0
      [ 52000 ns] Test is running, debug_wb_pc = 0xbfc00b9c
----[ 53145 ns] Number 8'd03 Functional Test Point PASS!!!
      [ 62000 ns] Test is running, debug_wb_pc = 0xbfc00c64
----[ 69075 ns] Number 8'd04 Functional Test Point PASS!!!
      [ 72000 ns] Test is running, debug_wb_pc = 0xbfc00690
      [ 82000 ns] Test is running, debug_wb_pc = 0xbfc00850
      [ 92000 ns] Test is running, debug_wb_pc = 0xbfc00828
      [102000 ns] Test is running, debug_wb_pc = 0xbfc00844
      [112000 ns] Test is running, debug_wb_pc = 0xbfc00870
      [122000 ns] Test is running, debug_wb_pc = 0xbfc0083c
      [132000 ns] Test is running, debug_wb_pc = 0xbfc00860
      [142000 ns] Test is running, debug_wb_pc = 0xbfc0082c
      [152000 ns] Test is running, debug_wb_pc = 0xbfc00850
      [162000 ns] Test is running, debug_wb_pc = 0xbfc00824
      [172000 ns] Test is running, debug_wb_pc = 0xbfc008a4
      [182000 ns] Test is running, debug_wb_pc = 0xbfc00924
      [192000 ns] Test is running, debug_wb_pc = 0xbfc009a0
----[196785 ns] Number 8'd05 Functional Test Point PASS!!!
      [202000 ns] Test is running, debug_wb_pc = 0xbfc00a00
----[211905 ns] Number 8'd06 Functional Test Point PASS!!!
      [212000 ns] Test is running, debug_wb_pc = 0xbfc00a94
=====
Test end!
----PASS!!!
```

图 4.2: lab14 测试结果

第一个测试点 debug 了大概大半天，超级辛苦，和之前不一样，这里在测试的时候并不显示流水线，都找不到对应指令，不好 de，不过最后找到了 `test.s` 文件，丢给 ai 帮我分析是什么地方的问题。主要是指导书上面的做了一些简化约束，不仔细读直接来做容易发现不了问题。

5 Cache 模块实现

接下来我们继续跟着指导书去完成 lab16 部分的 Cache 模块实现，完成 `cache.v` 的设计并通过 lab16 的测试。我们需要按照指导书的接口，实现了一个可综合的两路组相连 Cache。

5.1 总体设计与配置

按照指导书里面描述的，这里 Cache 采用如下结构与策略：

1. 两路组相连
2. 每路容量 4KB
3. Cache 行大小 16B（每行包含 4 个 32-bit word）
4. 写回 + 写分配
5. 伪随机替换策略（基于 LFSR）

在 `cache_top.v` 中，CPU 侧地址已被划分为 tag / index / offset 字段，其中，`index[7:0]`：地址 [11:4]，共 $2^8 = 256$ 组因此，每路 Cache 的容量为：

$$256 \times 16B = 4096B = 4KB,$$

5.2 Cache 存储阵列设计

Cache 内部包含两路数据阵列与对应的 tag / valid / dirty 元数据。数据阵列采用 Block RAM 实现，每行宽度为 128-bit；而元数据规模较小，使用分布式寄存器或分布式 RAM 更为合适。

Listing 7: Cache 数据阵列与元数据

```

1 (* ram_style = "block" *) reg [127:0] data_way0 [0:255];
2 (* ram_style = "block" *) reg [127:0] data_way1 [0:255];
3
4 (* ram_style = "distributed" *) reg [19:0] tag_way0 [0:255];
5 (* ram_style = "distributed" *) reg [19:0] tag_way1 [0:255];
6 (* ram_style = "distributed" *) reg valid0 [0:255];
7 (* ram_style = "distributed" *) reg valid1 [0:255];
8 (* ram_style = "distributed" *) reg dirty0 [0:255];
9 (* ram_style = "distributed" *) reg dirty1 [0:255];

```

为减少复位逻辑复杂度，Cache 采用 `initial` 块对所有行进行初始化，在配置 `bitstream` 后将 valid 与 dirty 位清零。该方式避免了在 `resetn` 时逐行同步清零带来的时序压力。

5.3 Cache 接口与握手语义

这里，Cache 与上层 CPU 通过 `cache_top.v` 进行连接，其接口行为遵循 **请求—响应式握手协议**。

1. 请求接收

CPU 使用 `memref_valid` 表示当前访存请求有效，Cache 使用 `addr_ok` 表示是否可以接收新请求。本实验中采用如下策略：

- 仅在空闲状态 (S_IDLE) 下拉高 addr_ok;
- 一旦接收请求, 立即拉低 addr_ok 并进入查找阶段;
- 任一时刻 Cache 内部仅处理一笔请求。

对应实现如下:

```
1 assign addr_ok = (state == S_IDLE);
```

2. 请求完成

当 Cache 完成一次访存事务时, 会拉高 data_ok 一个周期; 对于读请求, 在同一周期给出 rdata; 而对于写请求, 仅表示事务完成。

Cache 在统一的 S_RESP 状态完成响应, 保证接口行为的一致性。

5.4 Cache 状态机设计

Cache 的整体控制逻辑由一个有限状态机完成, 其主要状态包括:

Listing 8: Cache 主状态机定义

```
1 localparam S_IDLE    = 3'd0; // 空闲状态, 等待 CPU 请求
2 localparam S_LOOKUP  = 3'd1; // Cache 查找
3 localparam S_WB_REQ  = 3'd2; // 行写回
4 localparam S_RD_REQ  = 3'd3; // 向内存发起读请求
5 localparam S_RD_WAIT = 3'd4; // 等待内存返回数据
6 localparam S_REFILL  = 3'd5; // 回填 Cache 行
7 localparam S_RESP    = 3'd6; // 向 CPU 返回结果
```

5.5 Cache 查找阶段 (S_LOOKUP)

在 S_LOOKUP 状态中, Cache 同时读取两路 Cache 行, 并比较 tag 与 hit 位判断是否命中。

Listing 9: 查找信号

```
1 wire hit0 = v0 && (t0 == req\_tag);
2 wire hit1 = v1 && (t1 == req\_tag);
3 wire hit  = hit0 || hit1;
4 wire hit\_way = hit1; // 命中路: 若两路都命中 (理论上不应发生), 这里优先 way1
5 wire [127:0] hit\_line = hit1 ? line1 : line0;
```

- 读命中: 根据 offset[3:2] 从 Cache 行中选取 32-bit 数据, 进入 S_RESP;
- 写命中: 按 wstrb 进行字节级合并写入, 并置 dirty 位。

Listing 10: 查找命中

```
1 S\_LOOKUP: begin
2   if (hit) begin
3     // 命中: 读请求直接返回; 写请求更新该路并置 dirty
```

```

4      if (!req\_op) begin
5          rdata <= pick\_word(hit\_line, req\_offset[3:2]);
6      end else begin
7          rdata <= 32'b0;
8      end
9
10     // 写命中：按 wstrb 做字节合并写，并将对应路 dirty 置 1
11     if (req\_op) begin
12         if (hit\_way) begin
13             data\_way1[req\_index] <= merge\_store(line1, req\_offset[3:2],
14                 req\_wstrb, req\_wdata);
15             dirty1[req\_index] <= 1'b1;
16         end else begin
17             data\_way0[req\_index] <= merge\_store(line0, req\_offset[3:2],
18                 req\_wstrb, req\_wdata);
19             dirty0[req\_index] <= 1'b1;
20         end
21     end
22
23     state <= S\_RESP;
24 end

```

命中路径不访问主存，延迟最小。

5.6 Cache 缺失与替换策略

当两路均未命中时，Cache 判定为 Cache Miss，并选择一路作为 victim。

Listing 11: 查找缺失

```

1  else begin
2      // 不命中：锁存 victim 信息
3      // - 若 victim 行 dirty，则先写回 (S\_WB\_REQ)
4      // - 否则直接发起读 refill (S\_RD\_REQ)
5      victim\_way <= sel\_way;
6      victim\_tag <= sel\_tag;
7      victim\_dirty <= sel\_dirty;
8      victim\_line <= sel\_line;
9      state <= sel\_need\_wb ? S\_WB\_REQ : S\_RD\_REQ;
10 end

```

5.6.1 替换策略

- 若存在 invalid 行，优先选择该行；
- 若两路均 valid，则使用 LFSR 生成的伪随机位选择替换路。

Listing 12: 替换策略


```

1 // victim 选择:
2 // - 优先选 invalid 的那一路 (避免替换有效数据)
3 // - 两路都 valid 时, 使用 lfsr[0] 伪随机选路
4 wire sel_way = (!v0) ? 1'b0 : (!v1) ? 1'b1 : lfsr[0];

```

5.6.2 脏块写回 (S_WB)

若 victim 行的 dirty 位为 1, 则必须先将其写回主存。写回地址与数据如下:

```

1 wr_addr = {victim_tag, req_index, 4'b0000};
2 wr_data = {8'hff, victim_line[119:0]};

```

写回完成后进入读缺失阶段。

5.7 Cache 行填充 (Refill)

5.7.1 读请求阶段 (S_RD_REQ)

Cache 向主存发起一次 burst 读请求, 请求完整 Cache 行。

```

1 S_RD_REQ: begin
2     if (rd_rdy) begin
3         // 读请求握手: rd_req 在该状态被拉高; rd_rdy=1 表示内存接受读请求
4         // 进入等待 4 beat 返回, 并清空 fill_buf
5         beat_cnt <= 2'b00;
6         fill_buf <= 128'b0;
7         state <= S_RD_WAIT;
8     end
9 end

```

5.7.2 等待返回 (S_RD_WAIT)

主存以 4-beat 方式返回数据, Cache 使用计数器逐拍拼装 128-bit Cache 行。当检测到 ret_last 或 beat 数达到 4 时, 认为数据接收完成。

```

1 S_RD_WAIT: begin
2     if (ret_valid) begin
3         // 依次接收 4 个 32-bit 数据, 拼装成 128-bit 的一整行
4         case (beat_cnt)
5             2'd0: fill_buf[31:0] <= ret_data;
6             2'd1: fill_buf[63:32] <= ret_data;
7             2'd2: fill_buf[95:64] <= ret_data;
8             2'd3: fill_buf[127:96] <= ret_data;
9         endcase
10
11         if (ret_last || beat_cnt == 2'd3) begin
12             // 最后一拍 (ret_last=1) 到达, 进入 refill 写入
13             state <= S_REFILL;
14         end else begin

```

```

15         beat\_cnt <= beat\_cnt + 2'b01;
16     end
17 end
18 end

```

5.7.3 行写入与写分配 (S_REFILL)

在 S_REFILL 状态:

- 将新行写入 victim 路;
- 设置 valid 位;
- 根据请求类型决定 dirty 位;
- 对写缺失请求执行一次合并写操作 (write-allocate)。

```

1 S\_REFILL: begin
2     // refill 完成:
3     // 1) 更新 LFSR (为下次替换提供伪随机位)
4     // 2) 将 fill\_buf 写入 victim 路
5     // 3) 若本次是写请求 (store miss), 则写入时把写数据 merge 进去, 并置 dirty
6     lfsr <= {lfsr[6:0], lfsr[7] ^ lfsr[5] ^ 1'b1};
7
8     // 写入数据阵列 + 更新 tag/valid/dirty
9     if (victim\_way) begin
10        data\_way1[req\_index] <= req\_op ? merge\_store(fill\_buf, req\_offset[3:2],
11            req\_wstrb, req\_wdata) : fill\_buf;
12        tag\_way1[req\_index] <= req\_tag;
13        valid1[req\_index] <= 1'b1;
14        dirty1[req\_index] <= req\_op;
15    end else begin
16        data\_way0[req\_index] <= req\_op ? merge\_store(fill\_buf, req\_offset[3:2],
17            req\_wstrb, req\_wdata) : fill\_buf;
18        tag\_way0[req\_index] <= req\_tag;
19        valid0[req\_index] <= 1'b1;
20        dirty0[req\_index] <= req\_op;
21    end
22
23    // 对 CPU 的读返回: 读 miss 在 refill 完成后返回 fill\_buf 对应 word
24    // 写请求无需返回数据
25    if (!req\_op) begin
26        rdata <= pick\_word(fill\_buf, req\_offset[3:2]);
27    end else begin
28        rdata <= 32'b0;
29    end
30
31    state <= S\_RESP;
32 end

```

5.8 写策略与字节写支持

5.8.1 Write-back 与 Write-allocate

本实验采用 write-back 策略，仅在 Cache 行被替换时写回主存；同时采用 write-allocate 策略，在写缺失时先分配 Cache 行再执行写操作。

5.8.2 字节写掩码处理

自检程序会使用非全 1 的 wstrb 测试字节写能力。为此，本实验实现了字节级写合并逻辑，仅覆盖使能的字节，其余字节保持原值。

5.9 请求锁存与单请求处理模型

本 Cache 设计一次仅处理一条请求。当 valid 与 addr_ok 同时为 1 时，CPU 请求被锁存至内部寄存器 req_* 中，后续所有状态机操作均基于该锁存请求完成。

Listing 13: CPU 请求锁存寄存器

```

1 reg      req_op;
2 reg [ 7:0] req_index;
3 reg [19:0] req_tag;
4 reg [ 3:0] req_offset;
5 reg [ 3:0] req_wstrb;
6 reg [31:0] req_wdata;

```

5.10 实验结果

这里直接放 lab16 的测试结果：

```

-----
index fc finished
index fd finished
index fd finished
index fe finished
index fe finished
index ff finished
=====
Test end!
----PASS!!!

```

图 5.3: lab16 测试结果

6 icache & dcache 集成 CPU

在完成两路组相连 Cache (`cache.v`) 的实现之后, 下一步工作是将其正确地集成到 CPU 系统中, 形成指令 Cache 与数据 Cache。尝试在保持原有类 SRAM 访存接口不变的前提下, 引入 I / D wrapper、Cache burst 通路以及 AXI 桥扩展, 最终实现支持 cached / uncached 动态选择的完整访存子系统。

但是在实验跑之前, 我们要确定我们的 TLB 加入 CPU 之后是否过本 lab 的测试案例, 否则就真的 debugde 不出都不知道问谁了, 这里直接放结果图:

```
[48002000 ns] Test is running, debug_wb_pc = 0xbfc003a8
[48012000 ns] Test is running, debug_wb_pc = 0xbfc0068c
[48022000 ns] Test is running, debug_wb_pc = 0xbfc003b8
[48032000 ns] Test is running, debug_wb_pc = 0xbfc0069c
----[48034965 ns] Number 8'd94 Functional Test Point PASS!!!
[48042000 ns] Test is running, debug_wb_pc = 0x9fc00cf8
=====
Test end!
----PASS!!!
```

6.1 总体结构概述

集成 Cache 后, CPU 的访存数据通路结构如下:

- `mycpu_core` 仍然对外发出两路 SRAM-like 接口:
 - 指令访存: `inst_sram_*`
 - 数据访存: `data_sram_*`
- Core 额外输出段属性信号: `inst_cached` 与 `data_cached`, 用于指示当前访问是否应经过 Cache。
- 对于 cached 访问, 请求进入 `icache_top` / `dcache_top`, 并通过 Cache burst 端口完成行填充 (linefill) 或写回 (writeback)。
- 对于 uncached 访问, 请求直接透传至 `transfer_bridge`, 保持原有单拍访问语义。
- 顶层模块 `mycpu_top` 负责在 ICache 与 DCache 之间仲裁唯一的一组 Cache burst 接口, 并将其连接至 AXI 总线。

该设计在功能上实现了完整的 I / D Cache 体系结构, 同时最大程度复用原有 CPU 与总线逻辑, 降低了整体修改复杂度。

6.2 段属性判定

6.2.1 设计动机

为了在顶层正确选择 cached 或 uncached 访问路径, CPU 必须显式给出当前访存请求的段属性。根据实验指导书上面的要求, 需满足以下规则:

- kseg0 固定为 Cached
- kseg1 固定为 Uncached
- 其余虚拟地址段：TLB 命中且有效时，根据 TLB 的 C 位决定是否 cached

6.2.2 实现方式

在 mycpu_core 中新增 inst_cached 与 data_cached 输出端口，并基于虚拟地址高位与 TLB 属性生成段属性信号。该逻辑与地址翻译路径相互独立，仅用于 Cache / bypass 的选择。

Listing 14: 段属性判定逻辑

```

1 assign inst_cached = inst_kseg0 ? 1'b1 :
2               inst_kseg1 ? 1'b0 :
3               (inst_tlb_ok && (tlb_s0_c != 3'b010));
4
5 assign data_cached = data_kseg0 ? 1'b1 :
6               data_kseg1 ? 1'b0 :
7               (data_tlb_ok && (tlb_s1_c != 3'b010));

```

通过该方式，kseg0 始终以 cached 方式访问，保证了指令和数据在直映射段中具备 Cache 加速效果。

6.3 ICache / DCache Wrapper 设计

6.3.1 设计目标

由于 cache.v 对 CPU 侧暴露的是 SRAM-like 接口，但系统还需要在 cached 与 uncached 访问之间动态切换。因此在 Cache 与 CPU 之间引入 ICache / DCache Wrapper，它需要根据 *_cached 信号选择 cached 或 bypass 路径；并将 uncached 请求直接透传至 transfer_bridge；还要正确处理两种返回路径的时序关系，避免死锁。

6.3.2 uncached 返回不可门控问题

在实现过程中，一个关键注意点是：**uncached bypass 返回的 data_ok 不能被门控。**

原因在于：uncached 返回由 transfer_bridge 内部 FIFO 驱动，若 wrapper 根据当前 cached/bypass 状态去门控 data_ok，可能导致返回已从 FIFO 弹出但未送达 CPU，从而造成 CPU 永久等待、系统死锁。

因此，我们遵循如下原则：

- uncached 返回优先直通，不受 cached 状态影响；
- 在 bypass 返回与 cache 返回同周期到达的极端情况下，使用 1-deep hold buffer 暂存 cache 返回数据。

6.3.3 ICache Wrapper 关键逻辑

Listing 15: icache_top 返回选择逻辑

```

1 wire bypass_ok = uc_inst_sram_data_ok;
2 wire use_bypass = bypass_ok;
3 wire use_hold   = !use_bypass && hold_valid;
4 wire use_cache  = !use_bypass && !hold_valid && cache_data_ok;
5
6 assign inst_sram_data_ok = use_bypass || use_hold || use_cache;
7 assign inst_sram_rdata   = use_bypass ? uc_inst_sram_rdata :
8                               use_hold   ? hold_rdata :
9                               cache_rdata;

```

6.3.4 DCache Wrapper 特点

DCache 的整体结构与 ICACHE 类似，但需要额外支持写请求进入 Cache，并依赖 `cache.v` 内部的 write-back 与 write-allocate 机制完成存储操作。uncached store 仍通过 bypass 通路直接写主存。

6.4 Cache Burst 仲裁与顶层集成

实验指导书要求 Cache 以 burst 方式访问主存，以支持一次性完成整条 Cache 行的填充 (linefill) 与写回 (writeback)。在本实验中，由于系统中 ICACHE 与 DCache 共用同一组 AXI Cache burst 接口，若不加控制，可能会出现多路 Cache 同时发起 burst 请求、返回数据交叉等问题，从而导致功能错误甚至系统死锁。

因此，有必要在 CPU 顶层对 ICACHE 与 DCache 的 Cache burst 请求进行统一仲裁，确保任意时刻 AXI 总线上仅存在一条完整、连续的 burst 事务。

设计中采用如下策略：

- DCache 具有更高优先级（避免阻塞 store）；
- 一旦某个 Cache 的 burst 读请求被接受，在整条 burst 完成前不切换 owner；
- 通过状态寄存器显式记录当前 burst 归属。

6.4.1 仲裁状态机

Listing 16: Cache burst 仲裁

```

1 // -----
2 // Shared cache burst arbitration (I$ vs D$)
3 // -----
4 reg [1:0] cache_owner; // 0:none 1:icache 2:dcache
5 reg      cache_rd_inflight;
6
7 wire ic_need = ic_rd_req || ic_wr_req;
8 wire dc_need = dc_rd_req || dc_wr_req;
9
10 always @(posedge aclk) begin
11     if (!aresetn) begin
12         cache_owner <= 2'd0;

```

```

13     cache_rd_inflight <= 1'b0;
14 end else begin
15     if (!cache_rd_inflight) begin
16         if ((cache_owner == 2'd1) && ic_rd_req && cache_rd_rdy) begin
17             cache_rd_inflight <= 1'b1;
18         end else if ((cache_owner == 2'd2) && dc_rd_req && cache_rd_rdy) begin
19             cache_rd_inflight <= 1'b1;
20         end
21     end else if (cache_ret_valid && cache_ret_last) begin
22         cache_rd_inflight <= 1'b0;
23     end
24
25     if (!cache_rd_inflight) begin
26         case (cache_owner)
27             2'd0: begin
28                 if (dc_need) cache_owner <= 2'd2;
29                 else if (ic_need) cache_owner <= 2'd1;
30             end
31             2'd1: begin
32                 if (!ic_need && dc_need) cache_owner <= 2'd2;
33                 else if (!ic_need && !dc_need) cache_owner <= 2'd0;
34             end
35             2'd2: begin
36                 if (!dc_need && ic_need) cache_owner <= 2'd1;
37                 else if (!dc_need && !ic_need) cache_owner <= 2'd0;
38             end
39             default: cache_owner <= 2'd0;
40         endcase
41     end
42 end
43 end
44
45 assign cache_rd_req = (cache_owner == 2'd1) ? ic_rd_req : (cache_owner == 2'd2) ?
    dc_rd_req : 1'b0;
46 assign cache_rd_type = (cache_owner == 2'd1) ? ic_rd_type : dc_rd_type;
47 assign cache_rd_addr = (cache_owner == 2'd1) ? ic_rd_addr : dc_rd_addr;
48
49 assign cache_wr_req = (cache_owner == 2'd1) ? ic_wr_req : (cache_owner == 2'd2) ?
    dc_wr_req : 1'b0;
50 assign cache_wr_type = (cache_owner == 2'd1) ? ic_wr_type : dc_wr_type;
51 assign cache_wr_addr = (cache_owner == 2'd1) ? ic_wr_addr : dc_wr_addr;
52 assign cache_wr_wstrb = (cache_owner == 2'd1) ? ic_wr_wstrb : dc_wr_wstrb;
53 assign cache_wr_data = (cache_owner == 2'd1) ? ic_wr_data : dc_wr_data;
54
55 assign ic_rd_rdy = (cache_owner == 2'd1) ? cache_rd_rdy : 1'b0;
56 assign ic_wr_rdy = (cache_owner == 2'd1) ? cache_wr_rdy : 1'b0;
57 assign dc_rd_rdy = (cache_owner == 2'd2) ? cache_rd_rdy : 1'b0;
58 assign dc_wr_rdy = (cache_owner == 2'd2) ? cache_wr_rdy : 1'b0;
59

```

```

60 assign ic_ret_valid = (cache_owner == 2'd1) ? cache_ret_valid : 1'b0;
61 assign ic_ret_last  = (cache_owner == 2'd1) ? cache_ret_last  : 1'b0;
62 assign ic_ret_data   = cache_ret_data;
63
64 assign dc_ret_valid = (cache_owner == 2'd2) ? cache_ret_valid : 1'b0;
65 assign dc_ret_last  = (cache_owner == 2'd2) ? cache_ret_last  : 1'b0;
66 assign dc_ret_data   = cache_ret_data;

```

该机制保证了 AXI burst 传输的原子性，避免不同 Cache 的返回数据交叉。

6.5 AXI Bridge 对 Cache Burst 的支持

6.5.1 设计扩展

transfer_bridge 在原有 inst/data 单拍访问的基础上，新增对 Cache burst 的支持：

- Cache 读：4-beat burst (arlen=3)，用于行填充；
- Cache 写：4-beat burst (awlen=3)，用于写回；
- 使用独立的 CACHE_ID 区分 AXI 返回通路。

6.5.2 避免 Cache 返回被反压

为避免 Cache burst 返回被 inst/data FIFO 反压阻塞，当 AXI 返回 ID 为 CACHE_ID 时，rready 与 bready 被强制拉高，确保 Cache 能完整接收一整行数据。

6.6 关键点总结

集成过程中，需特别注意以下问题：

- uncached bypass 返回不可门控；
- Cache burst 返回不能被 inst/data FIFO 间接阻塞；
- burst 进行中禁止切换 ICache / DCache owner；
- kseg0 固定 cached，保证直映射段性能。

至此，CPU 成功集成了完整的 I / D Cache 。

6.7 实验结果

但是很可惜的是，做了一个星期，还是没能完整过完 94 个测试点，我在写报告回去的时候才留意到，指导书中说要修改 cache 模块来贴合 uncached 的情况，但是已经到期末死到临头了，已经没有太多时间来做，尝试用 ai 帮忙改了一下这里也还是不对。这里直接放一个结果截图：


```

-----[19288985 ns] Number 8'd68 Functional Test Point PASS!!!
      [19292000 ns] Test is running, debug_wb_pc = 0x9fc00dbc
      [19302000 ns] Test is running, debug_wb_pc = 0xbfc00684
-----

[19304017 ns] Error!!!
      reference: PC = 0x9fc1e898, wb_rf_wnum = 0x12, wb_rf_wdata = 0x00000001
      mycpu      : PC = 0x9fc1e894, wb_rf_wnum = 0x12, wb_rf_wdata = 0x00000001
-----

```

图 6.4: idcache 测试结果图

6.8 性能实验分析

想去分析一下 icache 和 dcache 的性能，但是观察测试点的波形图看不是很懂，这里我们另外设计一个 testbench 文件去测试一下命中率情况，我们让 ai 生成 5000 条指令，读写的比例为 70%，然后依次输入 cpu 中。

由于 Cache 采用单 outstanding 设计，如果在一次访问过程中触发了 burst 读请求，则该访问被判定为 miss；否则判定为 hit。测试文件分别统计 ICache 与 DCache 的访问次数、命中次数与缺失次数，并计算对应的命中率。此外，对 DCache 还额外统计写缺失及脏块写回次数。

相关命中率统计结果如图所示。

```

==== Dual-cache hitrate report ====
N=5000 IR=70%
I$: total=3663 hit=3471 miss=192 hitrate=94.76%
D$: total=1337 hit=627 miss=710 hitrate=46.90%
D$: read total=667 miss=356 | write total=670 miss=354 | writeback(line)=341
=====

```

从结果中可以看到，I-cache 的命中率达到 94.76%，说明指令访问具有良好的时空局部性，当前 I-cache 结构能够有效支持程序的取指需求，对整体性能影响较小；相比之下，D-cache 的命中率仅为 46.90%，且读写访问均存在较高的缺失率，并伴随频繁的写回操作，表明数据访问过程中缓存失效率较高，D-cache 已成为系统性能的主要瓶颈，后续可通过优化数据访问模式或调整缓存结构参数加以改进。

7 实验总结

本次实验做的比较漫长，一方面是内容很多，实验指导书上面有很多实现的细节，但是总是漏掉一部分，导致 debug 的时间很漫长，而且这次实验的测试点并不好 debug，波形图很多信息其实看不太懂，这也跟原理部分和这个新的 cpu 框架理解不透彻有关，感觉很多错误都集中在犹豫周期的原因，一些信号上面不是立即更新，导致死锁之类的；另一方面跑一次编译需要二十多分钟，相当折磨。

但是仍然收获满满，在实现之中对 cache 和 TLB 的实现细节更加理解透彻了，但实现之中仍然有很多实现的小 trick，无论是实验指导书上讲述的简化设计还是在实现过程中的方便，比如一些冲突直接用阻塞处理。这些详细的设计留给以后有机会的学习吧。

然后代码已开源在仓库链接:https://github.com/Absurdaaa/NKU_CS_courses/tree/main/%E8%AE%A1%E7%AE%97%E6%9C%BA%E4%BD%93%E7%B3%BB%E7%BB%93%E6%9E%84/lab11-17/rtl

最后感谢董老师一次又一次的延迟提交日期，给我们繁忙的大三时间留点空气。